Angular's *Async* Secret Weapon : **RxJS**

filter Async Data Observables

Multicasting Event Streams

debounceTime Subscriptions

Operators map Angular Integration

fromEvent Streams catchError

Niyati Kaneriya

Why RxJS ?

- **Async made easy**: No more callback hell or tangled Promises.
- **Reactive by design**: Handle streams of data, user events, and HTTP calls with elegance.
- **Everywhere in Angular**: RxJS powers HTTP, forms, routing, and more.

If you’re not using RxJS, you’re missing out on Angular’s true power

RxJS in Action

```
// 1. Map: Double each number in an array
of(1, 2, 3).pipe(
  map(num => num * 2)
).subscribe(console.log); // Output: 2, 4, 6

// 2. Filter: Only even numbers
of(1, 2, 3, 4).pipe(
  filter(num => num % 2 === 0)
).subscribe(console.log); // Output: 2, 4

// 3. DebounceTime: Wait for user to stop typing
searchInput.valueChanges.pipe(
  debounceTime(300)
).subscribe(value => console.log('User stopped typing:', value));
```

- **map**: Transform each value
- **filter**: Only what matters
- **debounceTime**: Smooth user input

What's your favorite RxJS operator? Share below!

Must-Know Operators

Operator	What It Does	Use Case
map	Transform data	Format API responses
filter	Filter by condition	Ignore empty input
switchMap	Switch to new stream	Live search
debounceTime	Delay emissions	User typing
catchError	Handle errors	API failures

RxJS Best Practices for Angular 18

- **Chain, don't nest:** Use .pipe() for readable, maintainable streams.
- **Let Angular clean up:** Use the AsyncPipe in templates to manage subscriptions automatically.
- **Strong typing:** Leverage TypeScript for safer, smarter code.
- **Avoid memory leaks:** Combine takeUntil with DestroyRef for subscription cleanup.
- **Handle errors up front:** Use catchError and retry for robust flows

RxJS + Signals?

- Angular's new **Signals** are hot, but RxJS is still essential for:
  - ↠ Event streams (user input, WebSockets)
  - ↠ Complex async flows
  - ↠ Combining multiple data sources
- Use Signals for UI state, stick with RxJS for streams.

RxJS + Signals = maximum firepower

Pro Moves

- **Optimize performance:** Use AOT compilation for faster loads.
- **Modern state management:** Pair RxJS with NgRx, Akita, or NGXS for scalable apps.
- **Clean code:** Avoid manual subscriptions—let Angular and RxJS do the heavy lifting.
- **Stay up to date:** Angular 18+ brings modularized operator imports and better config via app.config.ts

What's the RxJS pattern that leveled up your Angular projects?